PROJECT PROPOSAL

Financial Statement Simulator

An auditable, standard-neutral simulator of corporate financial statements, with a validated reference implementation on real US GAAP and IFRS filings

Prepared by Jaebum (Albert) Chung | Prepared for Chak | Status: full proposal | July 2026

Internship window: 31 August 2026 to 26 February 2027

1. Executive summary

This proposal specifies an enterprise-grade financial statement simulator: a system that reads a company's annual report, re-expresses it in one state space spanning both accounting standards, advances that state under a macro and industry scenario with an accounting engine that cannot break the books, and renders next-period statements in the company's own line items, under the company's own accounting standard. The design targets the uses a large bank has for such a machine: forward-looking credit assessment, scenario and stress analysis of counterparties and clients, portfolio-wide what-if screening, and a controlled analytical substrate that model risk management can audit.

Four design commitments distinguish the system from a spreadsheet model or a licensed data feed.

1. *Ingestion is held to a near-zero error bar* enforced by construction: several decorrelated readers extract every figure independently, a field is accepted only when independent paths agree exactly, and accounting identities arbitrate what survives; everything else is flagged for review, never guessed.
2. *The state-space encoding is provably lossless*: encode-then-decode reproduces the source statement exactly, cell for cell, because the encoder is injective (no two distinct statements share an encoding) and the decoder recomputes exactly the values the accounting identities pin down.
3. *The simulation cannot silently break accounting*: every projected stock movement is carried by an explicit, balanced flow; cash is never a residual; the engine asserts the identities and rejects any path that violates them.
4. *The machinery is written once*: one union state space spans both standards and every firm; behavior (how a line item moves under a scenario) attaches to a small vocabulary of authored economic roles called the common behavior layer; the engine and the scenario layer therefore serve every name in a portfolio with no per-firm or per-standard code.

All four commitments are already demonstrated: a validation build accompanying this proposal ran the full pipeline on four real annual reports from two US GAAP filers (Apple, Microsoft, Form 10-K) and two IFRS filers (SAP, Spotify, Form 20-F): **994 of 994** accepted PDF-extracted cells match the filers' own tagged values exactly (zero errors, zero silent misses, every discrepancy source explained); reconstruction through the state space is exact on **1,162 of 1,162** cells; and Monte Carlo simulation under six scenarios produces internally consistent statements on every path, moving in the economically required direction on every directional test. Section 11 reports the full results. Each design commitment has its mechanism section: the reader-reconciliation gate behind the ingestion bar (Section 7), the encoding theorem (Section 8.1), the balanced-flow engine (Section 9), and the common behavior layer over the union state space (Section 8.2); around them sit the scenario layer (Section 10) and the testing and governance wrappers that make the system deployable inside a bank (Sections 12 and 13).

Contents

2. Context and alignment

The enterprise-grade bar for this project has been the intent from the outset. Prior intern contributions (Hong Xiao and Elton) established the core of the simulator: an internally consistent, no-circularity accounting engine (no computed value depends on itself) and autoregressive driver modeling validated on synthetic data. What the project did not yet have, and what this proposal scopes, are the pieces the system hinges on: an extraction layer that reads real filings (US GAAP and IFRS) into structured statements, per-standard encode and decode adapters that map those statements to and from one state space, the union state space itself, and a credible relationship layer that maps scenarios to the flow variables.

The work divides into two parts. **Part I**, the MVP and the focus of this proposal, is a faithful simulator: it reads a real filing into a structured statement and produces next-period statements that carry the same line items, comply with the source standard, and move in the right direction under scenarios. Its driver layer is a simple state-space model, stochastic by construction, so Part I already runs stochastic simulations: for a given scenario it samples driver noise and propagates it through the engine to a distribution of internally consistent statements. **Part II**, a later and conditional extension, keeps the same machinery but replaces the simple noise with a learned, conditional joint distribution of the drivers (the main modelling IP). In the phased plan of Section 14, Part I is Phases 0 through 3 and Part II is Phase 4.

Working code has already retired the riskiest mechanical claims. A one-day knowledge-graph spike proved the taxonomy-graph encoding on one balance sheet; the validation build described in Section 11 extended that proof to all three core statements, four firms, two accounting standards, a PDF-only extraction mode with a measured error rate, exact reconstruction, and closed-book simulation under scenarios. The proposal can therefore commit to acceptance gates with observed evidence that they are reachable, rather than engineering judgment alone.

3. Problem statement and intended use

A bank consumes corporate financial statements at scale: in credit underwriting and annual reviews, counterparty monitoring, leveraged-finance covenant design, equity and debt research, and client advisory. Nearly all of that consumption is retrospective. The forward-looking questions that matter (what do this client's statements plausibly look like next year if rates stay high, if their industry's pricing tightens, if a recession arrives) are answered today by hand-built spreadsheet models: slow to build, inconsistent across analysts, unauditable in aggregate, and quietly dependent on plugs that make balance sheets balance by fiat.

The simulator addresses that gap with a system that is (i) *faithful*: it starts from what the company filed, to the digit; (ii) *consistent*: every simulated statement satisfies the accounting identities exactly, with no plugged balancing items; (iii) *comparable*: one union state space spans firms and standards, with behavior attached to a common vocabulary above it, so the same scenario can be pushed through every name in a portfolio; (iv) *presentable*: outputs render in each firm's own statement lines, so an analyst or an accountant reviews familiar statements, not a model's internal schema; and (v) *auditable*: every number in every output traces to inputs, rules, and seeds that reproduce it bit for bit.

Intended users and uses. Credit and research analysts (single-name scenario statements), portfolio and risk teams (batch what-if screening under common scenarios), and stress-testing functions (a consistent counterparty-level complement to top-down models). The MVP is an analytical tool, not a regulatory model; the governance posture (Section 13) is nevertheless built so that graduation into a governed model inventory is a process step, not a rewrite.

Non-goals. The system does not forecast point estimates competitively with equity research, does not price securities, and does not attempt cross-standard conversion (a statement read under IFRS is simulated and rendered under IFRS). Part I's stochastic output is a scenario-conditioned distribution with reasoned dynamics, not a calibrated probabilistic forecast; calibration is exactly the Part II extension.

4. Prior work

Internal. Three internal lines of work feed this design directly, and the reference implementation adopts concrete machinery from each. The prior interns' simulator frameworks contribute the *symbolic-first* discipline: financial models declared as SymPy equation systems, validated before any numerics (dependency-graph acyclicity, completeness, and the accounting identity proven by symbolic cancellation), then compiled into batched TensorFlow pipelines; this proposal's engine adds exactly that verification layer (Section 9) and runs its Monte Carlo in TensorFlow the same way. The author's prior LLM-extraction pipeline contributes the LLM calling logic (a gateway client with patient retry and tolerant JSON unwrapping), the page-identification and schema-constrained extraction prompt patterns, and the repeated-run median-vote hallucination control, all reused verbatim in the gated LLM stages of Section 7; its forecasting layer is an independent TensorFlow implementation of the Vélez-Pareja cash-budget ordering whose no-circularity treatment this engine matches. The director's extraction tooling established the redundant-reader pattern that remains the robustness backbone. A one-day spike (June 2026) validated the knowledge-graph encoding mechanics on a real balance sheet [20], and the validation build accompanying this proposal extended it end to end (Section 11).

Statement modeling without plugs. The requirement that projected statements balance by construction rather than by a plug has a literature. Vélez-Pareja and Tham develop forecasting schemes with no plugs and no circularity, showing that a balancing item computed as a residual conceals errors that double-entry construction exposes [1]. Stock-flow consistent macro modeling (Godley and Lavoie) applies the same discipline economy-wide: every flow comes from somewhere and goes somewhere, and accounting matrices must close [2]. Penman's financial statement analysis frames the articulation of income, balance sheet, and cash flow that the engine enforces mechanically [3]. The engine in this proposal is the firm-level, line-item-level application of those principles, with the closure asserted programmatically on every simulated path.

Structured financial data and taxonomies. XBRL 2.1 and its calculation linkbases (the filer-published subtotal arithmetic) define machine-readable statement structure [4], extended by the newer Calculations 1.1 specification [5]; the FASB US GAAP taxonomy [6] and the IFRS Accounting Taxonomy [7] supply the concept ontologies (roughly 17,000 standardized line-item definitions each) that this proposal uses as the state space's node set. The SEC has required interactive-data filings since 2009 [8] and inline XBRL for operating companies since 2019 per the EDGAR Filer Manual [9]; ESMA's ESEF imposes the same for EU issuers [10]. The open-source Arelle platform [11] is the reference processor for these artifacts and is used by regulators; it is the tag-path reader here, the path that consumes the filer's own machine-readable tags rather than the rendered document.

Document extraction. Table structure recognition from rendered documents is an active field with dedicated datasets and models (for example PubTables-1M [12]); financial-domain question answering datasets such as FinQA [13] and TAT-QA [14] document how error-prone numerical extraction from filings is for learned systems, which motivates this proposal's decorrelated-reader gate and identity checks rather than trust in any single reader, learned or deterministic.

Vendor data. Standardized products (Compustat and peers) reclassify native line items into fixed templates: a non-injective encoding that discards the presentation this system must reproduce, so they cannot serve as the primary representation regardless of accuracy. As-reported vendor feeds preserve native lines but are themselves extraction pipelines with their own error rates, coverage, latency, and licensing constraints. Most decisively, the capability under examination is ingestion of an arbitrary born-digital annual report (authored electronically rather than scanned; Section 7 makes the boundary precise); outsourcing it fails the test by construction. A licensed feed remains useful as an external adjudication reader (Section 7).

Governance standards. The design aligns to SR 11-7's model-risk expectations (documented methodology, effective challenge, outcome analysis) [16] and to BCBS 239's risk-data principles (accuracy, integrity, completeness, timeliness, adaptability) [17], which is what "enterprise-grade" concretely means in Section 13.

5. Definition of success (acceptance bar)

Given a prior-period statement and a scenario, the system produces simulated next-period statements that:

- an accountant cannot identify as obviously wrong;
- preserve the same field set as the input statement (exact line-item parity);
- conform to the applicable standard (US GAAP or IFRS), matching the source filing; and
- respond in the correct direction to scenario changes.

Quantified gates. The gates below bind the MVP, and every one of them has already been met by the validation build on its four-firm set (Section 11):

1. **Extraction.** On the gating set (the fixed list of filings the gates are measured on; Section 12 sets its composition), the PDF-only mode (tag path withheld) must show *zero* errors among accepted cells against verified ground truth (the filer's own tagged facts, or hand-verified double-entry where the filing carries no tags), with every non-accepted cell flagged rather than silently dropped, and every unmatched ground-truth row assigned to a documented benign category: a tagged value with no cell to read on the statement face (either a figure the filer prints inside a label's own text, or a subtotal the filer leaves unprinted, verified through its children). The measured claim is reported with its rule-of-three bound, which is nothing more than asking what error rates zero observed errors can rule out: if the true per-field error rate were p, zero errors among N independent cells occurs with probability $(1-p)^N$, which stays above 5% only for $p \leq 3/N$ (because $(1-3/N)^N \approx e^{-3} \approx 0.05$), so observing N accepted cells with zero errors bounds the rate above by $3/N$ at 95% confidence [18]. The architectural target for silent *concordant* error (all readers agreeing on the same wrong number and the identities failing to catch it) is of order 10^{-8} per field (Section 7); four filings cannot measure that directly, so it is defended by construction and by seeded-error testing, and reported honestly as such.
2. **Reconstruction.** $D(E(s)) = s$ exactly, on every cell of every statement of every firm in the set: values, labels, order, signs, units, and precision attributes.
3. **Arithmetic.** Every derived subtotal foots from its children within the decimals-based tolerance: a value tagged $\texttt{decimals} = d$ is stated to the nearest 10^{-d}, so each of n addends and the printed total carries up to half that unit of independent rounding, and the recomputed sum may differ from the printed one by up to $0.5 \cdot 10^{-d} \cdot (n+1)$; A = L + E holds; the cash flow ties to the balance-sheet cash delta.
4. **Simulation integrity.** Across all Monte Carlo paths of all scenarios, zero identity violations: the engine may not introduce any imbalance beyond the filer's own printed rounding residual, which it must preserve exactly.
5. **Directional battery.** Scenario means move with the required signs (Section 10); the battery must pass in full.

6. **Plausibility.** Sampled statements pass automated accountant-style bounds (growth, margin, non-negativity, continuity), and a qualified accountant review of representative statements finds nothing obviously wrong. Reviewer sign-off is the human gate on top of the automated battery.

Precision asymmetry. Extraction is held to the near-zero bar because a misread figure invalidates everything downstream. Encoding is exact by construction. Simulation output needs to be solid, robust, and directionally credible rather than precisely forecast; its correctness dimension is internal consistency, which is absolute.

6. System architecture

The simulation core is common to both standards; standard-specific work sits at the boundaries. Figure 1 shows the pipeline; Table 1 lists the components and their validation status. The exposition follows the pipeline: first the extraction boundary and its error model (Section 7), which produces the structured statements everything downstream consumes; then the state space those statements encode into (Section 8); then the engine and the scenario layer that evolve it (Sections 9 and 10).

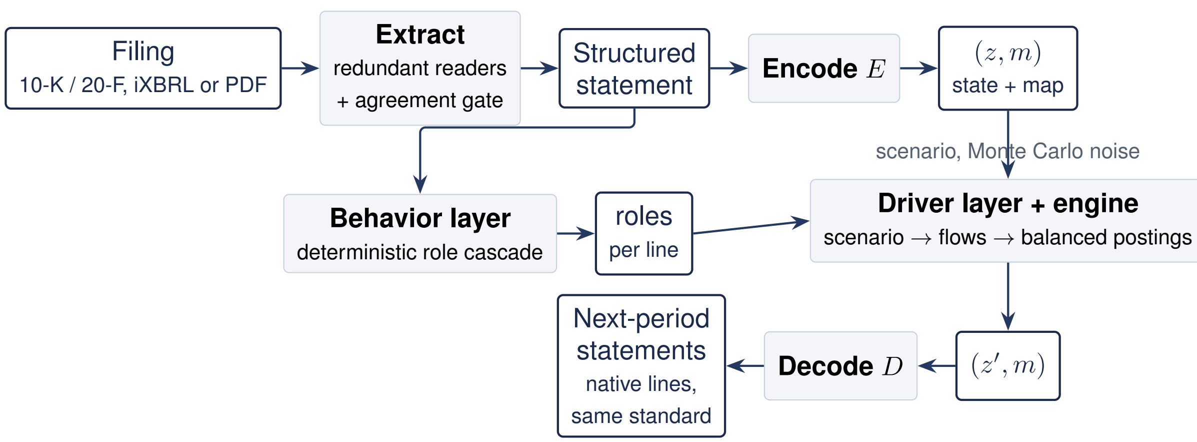

Figure 1. End-to-end pipeline. Extraction (reading the filing) and encoding (re-representing it) are distinct stages with different error models: extraction carries a measured error rate; E and D are deterministic and lossless. The presentation map m retained at encode time is what makes the output print in the firm's own line items. The behavior layer runs beside the encoder, not inside it: the deterministic cascade assigns each line its economic role, the driver layer and the engine consume those roles alongside (z, m), and the identities bind on (z, m) alone, so a misassigned role can steer a driver but never unbalance a book.

Component	Status	Role
Statement extractor	**Validated**	Reads a filing (tagged data or PDF) into a structured statement using decorrelated readers and an agreement gate; carries the near-zero read-error target.
Onboarding + mapping artifact	**Validated**	Per-document build step: ingestion with the model under the leash emits a signed, hash-bound artifact of label-to-concept choices and adjudications; the runtime replays it deterministically (Section 7.1).
Encode/decode adapters	**Validated**	Losslessly map structured statements to and from the union state space; perfect reconstruction proven per filing.
Union state space	**Validated**	One state space over both the US GAAP and IFRS taxonomies, specified as a knowledge graph with the common behavior layer as its authored annotation (Section 8.2); retains the per-firm presentation map.
Accounting engine	**Rebuilt + validated**	Evolves stocks from balanced flows; identities preserved by construction, asserted on every path (Section 9).
Driver-to-flow layer	**MVP validated**	Maps scenarios to flows with reasoned nonlinear responses and sampled noise (Section 10); the Part II learning slot.

Table 1. System components. “Validated” means exercised end to end by the four-firm validation build of Section 11.

7. Extraction robustness

Extraction is held to a far higher precision bar than the rest of the system because a misread figure invalidates everything downstream. The approach mirrors the engineering already used in the director’s own tools: several readers cross-check one another, and a field is accepted only when they agree.

What reading a PDF means, and why the readers differ. A born-digital PDF does not store text the way a plain-text file does. It stores drawing instructions: place this character shape, this glyph of this font, at these coordinates. Recovering a statement from those instructions is an act of interpretation, and every step of it can err: shapes map back to characters through per-font tables, word breaks are inferred from spacing (a space is usually a gap, not a stored character), and lines, rows, and columns are assembled from coordinates. The readers of Figure 2 are independently written interpreters of the same authored instructions, chosen so their mistakes do not overlap. R2 and R3 run one line grammar over two different text-extraction engines (pdfminer and pypdf), separate codebases that perform the inference differently and fail differently; the untagged sweep of Section 11 met both failure shapes in the wild, space glyphs that vanish and digits split inside a number, and each surfaced as reader disagreement. R1 trusts neither engine’s assembled lines: it reads raw word positions and reconstructs the table spatially, clustering the right edges of numeric tokens into column bands, so it sees the alignment the text engines cannot and misses in ways they do not. The filer’s instructions are exact at the source, only interpretation errs, and the interpreters err differently, so their exact agreement on a figure is evidence rather than repetition. R4 does not interpret the rendered page at all: when the filing carries tags, it consumes the filer’s machine-readable facts, with sign, scale, and period stated explicitly.

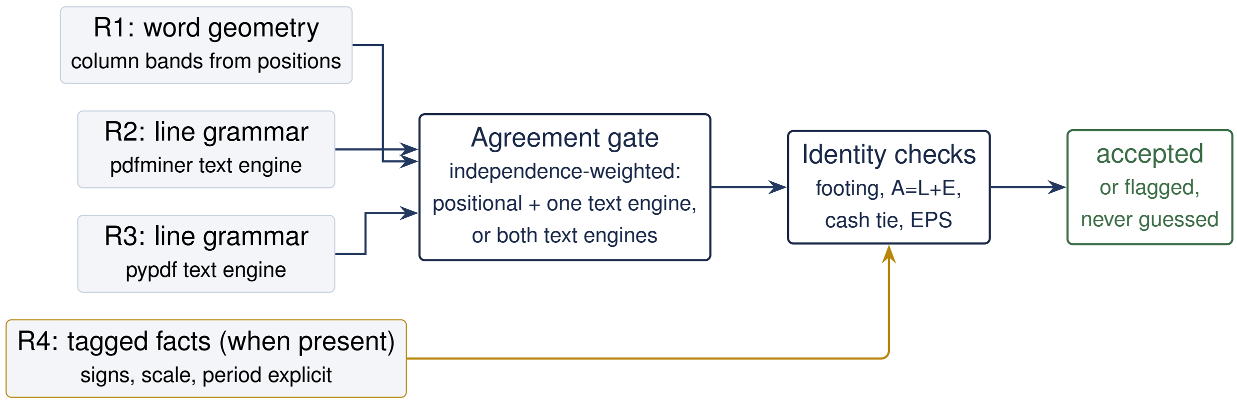

Figure 2. The reconciliation gate. Readers consume different representations of the same document; the gate weights agreement by path independence (the two line readers share a grammar, so their mutual agreement cannot outvote positional evidence); identity checks catch concordant errors that break arithmetic; tagged facts, when present, are the strongest reader and the ground truth for measuring the others.

Under this gating rule the two failure modes have very different costs: a disagreement costs review time; the expensive event is *silent concordance*, every reader returning the identical wrong value. The rate of that event has a clean two-part structure. Write p_i for reader i's per-field error rate and, given that reader i errs, write $q_i(v)$ for the probability that the wrong value it returns is v; with k genuinely independent readers, the error events are independent and so are the wrong values drawn. Silent concordance then requires two coincidences in sequence: every reader errs on the field at once, and the erring readers land on one identical wrong value. The first is the product of the error rates; the second is the overlap of the wrong-value distributions, a single joint quantity for the whole ensemble, because one reader has nothing to collide with. That overlap is the collision kernel c:

$$\Pr(\text{silent concordance}) = \left(\prod_{i=1}^{k} p_i\right) c, \qquad c = \sum_{v} \prod_{i=1}^{k} q_i(v). \tag{1}$$

The kernel enters once, not per reader, and it is small exactly when the readers fail differently: readers that share a failure mode concentrate their wrong values on the same candidates and inflate c. Three inputs set the scale of (1): observed single-reader error rates at or below the percent level bound the p_i; decorrelated failure modes keep the q_i spread over different wrong values, holding c down; and the identity checks shrink the collision sum, because a coincident value that breaks the printed arithmetic is caught downstream and only identity-consistent collisions remain. The design target is toward 10^{-8} per field, *measured* where measurement is possible and defended by construction where it is not. Three filters implement this:

1. **Cross-reader agreement** catches discordant errors; every disagreement is surfaced, never resolved silently. The acceptance rule is independence-weighted: the geometry reader plus either text-engine reader accepts; the two text-engine readers agreeing with each other, without positional support, is accepted only where geometry has no reading at all, and recorded as the weaker rule.
2. **Decorrelated input paths**: word positions; two different text extraction engines under one line grammar; the filer's tagged facts when present (explicit sign, scale, period, no dependence on visual layout); optionally a vision reader and the prior year's comparatives. Layout pathologies

(wrapped labels, note-reference columns, scale headers, glued currency amounts, unlabeled section totals) hit the layout readers and not the tag path; tag-path pathologies (extension tags, footnote-only values) do not afflict the layout readers.

3. **Accounting-identity checks** catch concordant errors that break arithmetic: footing along the calculation arcs, A $=$ L $+$ E, the cash tie, and per-share consistency, all within the decimals tolerance. They also catch genuine typos in the source filing, which are surfaced, since the extractor's ground truth is what the filing says.

Document scope: born-digital PDFs. The untagged mode requires statement pages to carry *authored* text, not merely present text: scanned or image-compiled documents, including OCR'd scans, are out of scope for the MVP. The distinction is about where the characters come from. On an authored page the characters are exact at the source: the filer's software placed every one, so only a reader's interpretation can err, and interpretation errors are what the reconciliation gate is built to catch. An OCR'd scan inverts this: the page is a picture, a recognition pass wrote its guesses at the characters into an invisible text layer (rendering mode 3) laid over the full-page raster, and a guessed digit is wrong before any reader runs. Such a scan passes a naive has-text check, but admitting it would feed every text-consuming reader from that single guess: the readers would agree as copies of one reading, not as independent checks, and agreement would stop being evidence. The gate is mechanical and per statement page: a page-dominating image XObject combined with absent or invisible text triggers the abstain rule, exactly as a missing mapping entry does (font embeddedness alone is deliberately not a hard signal, since authored documents set in the standard-14 fonts carry no embedded fonts). The reference implementation enforces and tests this: a scanned page and an OCR'd page seeded into an otherwise born-digital document each abstain with the scope error named. The cut removes an OCR dependency, not the correlated-failure concern: a born-digital file can still carry a corrupted text layer (broken ToUnicode maps, subset fonts mapping glyphs to wrong codepoints; one sweep document extracts prose but not a single numeral), and that corruption hits the layout and text readers together; the identity checks, the tag path where present, and any future pixel-consuming reader remain the defense there.

Tags are the best case, not a prerequisite. Where a document carries no tags (a private firm or another jurisdiction, provided the document is born-digital), the remaining readers still gate one another and the identities still bind; reduced redundancy surfaces as a higher flag rate, not silent error. Because the untagged mode is the one that generalizes beyond public filings, it is measured separately: the *PDF-only ablation* withholds the tag path from extraction entirely and scores the result against it. The ablation exists to certify the generalizing method, not to serve tagged filings: on a tagged filing the production extraction simply uses the tags, the best reader. Section 11 reports that ablation at zero errors on 994 accepted cells across the four filers, and an untagged sweep over fourteen arbitrary investor-relations annual reports (US GAAP and IFRS, five jurisdictions, banks and industrials, five distressed filers).

Arithmetic before semantics. The untagged pipeline validates before it interprets, on the principle that *words nominate, numbers confirm*: a thin label grammar proposes where structure might be, and a proposal survives only if the printed numbers close on it, in every reported column at once. Footing works this way: rows whose labels look total-like (a "total" prefix, the cash flow's "net cash provided by or used in" family, the income statement's customary subtotals, a row echoing its section's name, or an unlabeled row) nominate subtotal candidates, and the footing solver searches child weights in $\{+1, 0, -1\}$, accepting a tree only when every checkable column agrees; a wrong nomination simply fails to solve and is flagged, and several columns of real figures closing on the same wrong weight vector is a negligible coincidence, which is what makes the check safe without any semantic model. The balance-sheet identity anchors on canonical labels the same way: "total assets" against a combined right-hand total, or against "total liabilities" plus "total equity" (with a Chinese label pack for HK and PRC documents). It reports "not checkable" rather than guessing when an anchor is absent, which is how a netted bank presentation fails honestly instead of quietly. The cash tie finds the beginning, ending, net-change, and currency-effect rows by the same nomination, falls back to summing the three activity totals when no net-change row prints, and accepts either filer convention for where the currency effect sits; Microsoft's FY2024 render ties as beginning 34,704 plus change $-16{,}389$ against ending 18,315, in millions. All of this runs before any taxonomy concept is assigned. Concept mapping then attaches semantics to numbers that already foot, in the order of the next paragraph (the harvested lexicon first, a prior artifact when carrying, then a taxonomy label that names exactly one concept, then the model, under the polarity veto: a proposal whose debit or credit direction contradicts the printed sign is rejected); the encoder stores only what survived, and anything that fails a gate is flagged, never guessed, with a flagged balance-sheet cell refusing simulation outright. Figure 3 draws the order.

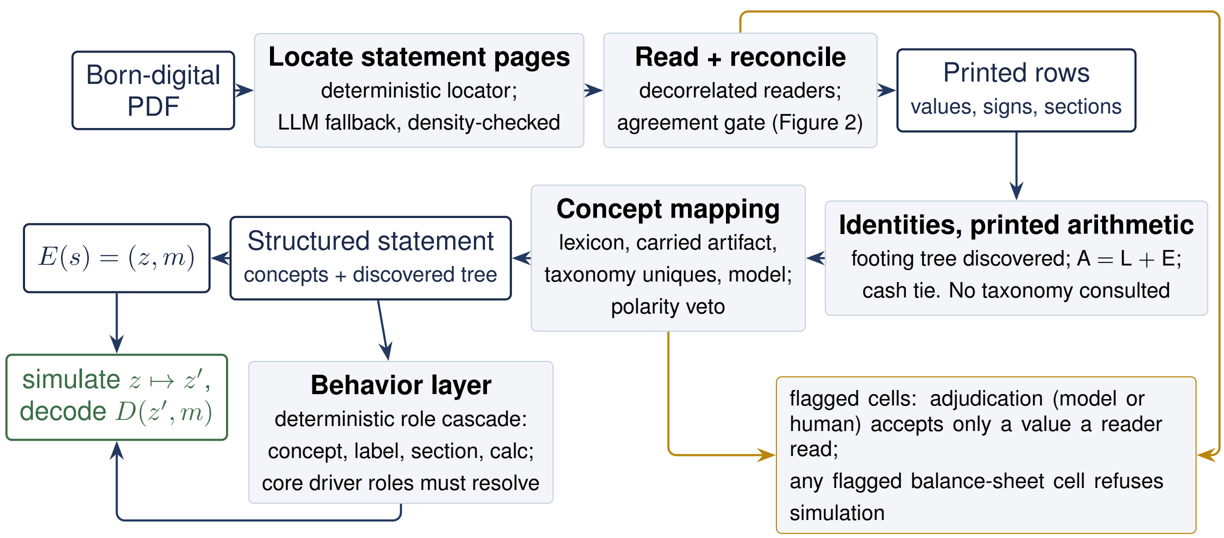

Figure 3. Untagged ingestion validates before it interprets. Printed rows that survive the reader gate are checked arithmetically first: the footing solver discovers the subtotal tree from the numbers themselves, and A = L + E and the cash tie bind on the printed totals, with no taxonomy consulted. Concept mapping then attaches semantics to validated numbers, the encoder stores what survived, and every failure is a flag, never a guess: adjudication can resolve a flag only into a value a deterministic reader read, and a flagged balance-sheet cell refuses simulation. The behavior layer joins at simulation time: the deterministic role cascade classifies every mapped line, and those roles are what the engine and the driver layer consume to advance the state; the identities and the encoding need nothing from them, and a firm whose core driver roles do not resolve is refused simulation with the missing roles named.

The mapping ladder. Concept mapping tries four rungs in order, each cheaper and more accountable than the next; a row is resolved by the first rung that succeeds. First the harvested lexicon: an exact match of the canonicalized printed label against labels the validation filers print, carrying the concepts their own tags declare, scoped per statement so that "Inventories" means the stock on a balance sheet and the period's delta on a cash flow. Second, when onboarding carries, the firm's own prior signed artifact: an exact match replays a reviewed choice. Third, the taxonomies' own standard labels under a uniqueness rule: the two standard setters name roughly 31,000 label forms and almost all of them name exactly one concept, so a printed label that matches one of those forms resolves deterministically (Bed Bath & Beyond's "Repayments of long-term debt" resolves to exactly that concept, agreeing with the filer's own tag), while the few dozen forms both taxonomies share ("Goodwill" exists in each) are dropped at load, so the tier can never assert an identity another standard concept shares a name with. In practice exact coincidences between printed wording and a standard label are rare, so this rung is a precision instrument rather than a coverage instrument: on the non-lexicon firm it resolved one row, correctly. Only the residue reaches the model, over a lexically retrieved shortlist; and both heuristic rungs (the taxonomy tier and the model) pass the balance-polarity veto, while the lexicon's and the artifact's exact matches are trusted.

Residual risk. What remains silent is the intersection: errors concordant across decorrelated paths, identity-preserving, and within rounding tolerance, for example a uniform mis-scaling that multiplies every value identically (identities are scale-invariant, so only the tag path or a share-consistency check catches it). The seeded-error battery deliberately injects the correlated pathologies (wrapped labels, scale exceptions, note-reference columns, digit transpositions, dropped signs), seeds a scanned page and an OCR'd page into an otherwise born-digital document and requires the born-digital abstain to fire (making the scope limit a tested behavior, not a sentence), and requires the gate to flag every one; a re-prompt of the same model on the same input is never counted as an independent check; and adjudication weight follows path independence, a standard-tag value outweighing two layout readers that agree with each other.

7.1 Onboarding and the mapping artifact

Ingestion as described so far is a per-run computation, and two of its ingredients are not: label-to-concept choices and flagged-cell adjudications are per-document judgment. Judgment gets a lifecycle rather than a rerun: made once with the model's help, validated mechanically, reviewed, signed, and then replayed deterministically for as long as the document's bytes do not change. Figure 4 draws that lifecycle; the three passages below give its rules. The asymmetry with the behavior layer of Section 8.2 is deliberate: sign-off gates concept mapping because it is per-document judgment, while versioning and the acceptance battery gate the behavior layer because it is firm-independent code.

The LLM's leash. LLM stages reuse the author's prior extraction pipeline wholesale: the calling logic, the batched page-identification prompts, the schema-constrained JSON extraction rules, and the repeated-run median-vote hallucination control. Their authority is deliberately narrow. Page identification is a fallback when the deterministic locator abstains; the artifacts measure that hierarchy: five fallbacks across the fourteen-document sweep (Section 11). Cell reading operates only on cells the reader gate flagged, and a flagged cell is accepted only when the LLM's median-voted value exactly matches one of the deterministic readers, so the LLM can break ties but can never introduce a number nobody read. Concept mapping chooses from a lexically retrieved shortlist and is vetoed by balance-polarity checks. When no endpoint is configured every stage degrades to its deterministic behavior and the corresponding cells simply remain flagged; a firm whose balance sheet carries flagged cells is refused simulation outright (a structural quality gate), which is precisely the review loop the abstain rule intends.

LLMs at build time, determinism at run time. The leash above binds *what* the model may say; a second, stricter rule binds *when* it may be consulted at all. Production ingestion must replay exactly to pass model approval, and a hosted model in the inference path cannot meet that bar: inference kernels are load-dependent in their low-order bits [15], greedy decoding amplifies a last-bit difference into a different answer, and hosted endpoints update silently, leaving

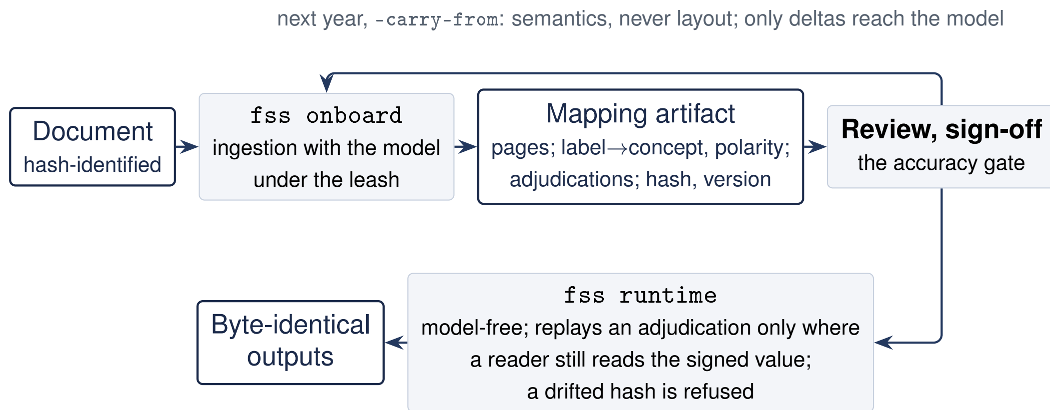

Figure 4. The onboarding lifecycle. `fss onboard` runs the full ingestion pipeline with the model enabled under the leash; every entry in the resulting mapping artifact is a survivor of the mechanical validators, and the artifact binds to the document's hash and carries a sign-off field. `fss runtime` never constructs a model client: it re-extracts deterministically, replays an adjudication only where a deterministic reader still reads the signed value, refuses a drifted hash with re-onboarding required, and reruns byte-identically. A prior year's signed artifact can seed the next year's onboarding: carry seeds semantics, never layout, and only genuine deltas reach the model and the reviewer.

no version to freeze. Accordingly the model is confined to change management. `fss onboard` runs ingestion with the LLM engaged and emits a *mapping artifact* per document: statement page locations, label-to-concept choices with balance polarity, and flagged-cell adjudications, each entry the survivor of the mechanical validators, with the source document's hash, the code version, and a sign-off field. A human reviews and signs the artifact; it is versioned alongside the code. `fss runtime` then never constructs a model client: it re-extracts deterministically, replays an adjudication only when a deterministic reader still reads the signed value today (the artifact can break ties but cannot inject numbers into a changed document), re-runs every check, logs the source, code, and artifact versions it ran under, and refuses a document whose hash has drifted with "re-onboarding required." The reference implementation demonstrates the property directly: two independent runtime executions over the same filings produce byte-identical outputs (twelve artifacts, zero differing hashes), the fourteen-document sweep reproduces its build-time results with zero model calls, and when the original hosted endpoint began refusing requests between build and demonstration, the runtime was indifferent: the artifacts were reconstructed from the committed build products without a model, which is the failure mode this split exists to absorb.

Across years: carry-forward. A mapping artifact binds to one document by hash: a signed document whose bytes change is refused, and a new fiscal year is a new document that must be onboarded. What keeps annual re-onboarding cheap is that a firm's line-item wording is sticky year over year, so `fss onboard -carry-from` seeds a filing's onboarding from the firm's prior signed artifact. Carry seeds semantics, never layout: the prior label-to-concept choices enter the resolution order between the lexicon and the model, so an exact-label match replays a reviewed choice instead of consulting anything, while page location stays with the deterministic locator (layout is a per-document fact) and adjudicated values never carry (they are year-specific by nature); only genuine deltas (lines the firm reworded, added, or restructured) reach the model and the reviewer. Carry replicates the prior artifact faithfully, errors included, so the sign-off it inherits from, not the mechanism, remains the accuracy gate. Section 11 demonstrates the mechanism on two adjacent-year pairs scored against the filers' own tags.

8. The union state space

Extraction delivers a structured statement in the filer's own lines; simulation needs a representation those lines can be evolved in. This section specifies that representation: first the encoding into it and the proof that nothing is lost, then the graph it is specified over and the behavior layer that makes one engine serve every firm.

8.1 Encoding, decoding, and perfect reconstruction

Why operate in a state space at all? First, line items are heterogeneous: firms present the same economics under different labels, orderings, and aggregations, so machinery written against native lines fragments into per-firm variants; the union state gives the simulation one stable schema. Second, standards become cheap to add: the driver layer and the engine wire once to the common behavior layer above the state (Section 8.2), so a new standard costs an adapter pair. Third, it is what makes Part II estimable: pooling firms requires a shared representation. The price of this indirection is that nothing may be lost on the way in or out.

Common means shared coordinates, not shared shape. A fixed common template (a vendor-style schema of standard slots) would force every firm's items through someone's judgment of equivalence and is non-injective by construction; this state space is instead the taxonomy graph itself, and z has no fixed width: each firm populates exactly the subset of nodes it discloses, at whatever granularity it discloses them. Nothing translates a firm's rows into foreign slots, because by the time the encoder runs, every row already carries its node: the filer's own tags put it there, or onboarding does (Section 7), and E then merely splits the resolved statement into values (z) and presentation (m). That boundary sits where it does deliberately: resolving a line to its node is fallible and therefore measured (Section 11), while E is proved, and housing every judgment upstream of the encoder is what keeps losslessness a proved property of E rather than an empirical claim about mapping quality. Firms of different shape still meet through three graph mechanisms, none of them per-firm:

- *Aggregation upward.* Different granularities reconcile through the calculation and hierarchy edges: a portfolio screen reads revenue for every name, as a recomputed member sum for a firm that discloses product and service components and as the leaf itself for a single-line firm.
- *Inheritance downward.* Drivers attach at concept level and inherit down the hierarchy to whatever leaves a firm has (Section 10). The mechanics matter here, because the natural worry is that driving an aggregate requires carving it up among the components. It does not, and the engine never does so: the driver pushes a *rate* down, each component leaf scales multiplicatively from its own base, and the aggregate is a derived row recomputed upward from the grown leaves. Two consequences fall out for free. Deltas distribute pro rata: a member holding 60% of revenue absorbs 60% of the dollar change, because equal rates on unequal bases produce contribution-weighted level changes. And footing is preserved exactly: the subtotal is re-derived from its children rather than driven and allocated, so there is no allocation residual to plug; driving the aggregate and distributing down would create precisely the rounding-and-residual questions this engine exists to avoid. The honest limitation is that a uniform rate preserves the disclosed mix: product and service shares stay constant under an aggregate-level scenario, so mix-shift dynamics need either a driver attached at the component level, which the graph supports, or the learned joint distribution of Part II, which is exactly the kind of structure it would learn.
- *Roles for the engine.* The authored role table and laws of motion are written once and are firm-independent: they wire behavior at the level the graph shares, and the calc-descendant

inheritance rule extends them to firm extensions automatically (Section 8.2).

The price of the union design is borne by the machinery, not the data: the engine and the driver layer walk a variable-width graph rather than index a fixed vector.

Write s for a structured statement: an ordered list of rows, each carrying a taxonomy concept, an optional dimensional qualification (axis and member, for statements that disaggregate a concept on their face), a native label, sign convention, unit and precision, and one value cell per reported column. The encoder outputs a pair

$$E(s) = (z, m), \tag{2}$$

where z carries the values of the information-bearing rows and m carries everything else: row order, labels, signs, units, precision, sections, column periods, and the derivation structure (calculation arcs and dimensional aggregation) discovered from the filing. The decoder renders $D(z, m) = \hat{s}$. Perfect reconstruction requires $D \circ E = \mathrm{id}$ on the domain of valid statements, which holds if and only if E is injective: no two distinct statements may share an encoding, or the decoder could not tell them apart.

Proposition 1 (Lossless reduction) *Let z store one value per (concept, dimensions, period-role) key (the period role separates presentations of the same concept, such as a beginning from an ending balance) for every row not derivable from the others, and let m store every remaining attribute of s together with the derivation rules below, where $v(r, p)$ is the value row r reports in column p, $\mathrm{children}(r)$ are the rows the filing's calculation arcs declare as summing into r with weights $w_{rc} \in \{+1, -1\}$, and $\mathrm{members}(r)$ are the dimension members that disaggregate r on the face:*

$$v(r,p) \;=\; \sum_{c \,\in\, \mathrm{children}(r)} w_{rc}\, v(c,p) \qquad \textit{or} \qquad v(r,p) \;=\; \sum_{r' \,\in\, \mathrm{members}(r)} v(r',p) \tag{3}$$

for calculation parents and dimensional aggregates respectively. Then E is injective and D as defined by re-rendering m and recomputing (3) *is a left inverse of E.*

The proof is by construction: two statements agreeing on (z, m) agree on every stored cell and every attribute, and their derived cells are equal because (3) is a function of stored cells. The only values dropped are exact functions of kept ones, so the reduction is lossless. Two lessons from the validation build sharpen the "key" clause, and both are now regression tests. *Dimensions:* Apple and Microsoft disaggregate revenue and cost of sales by product/service axis on the face, so the same concept legitimately appears in several rows; keys must carry the dimensional qualification. *Period roles:* a cash flow statement displays the same cash concept twice, as beginning and ending balances distinguished only by the preferred-label period role; collapsing them breaks injectivity, exactly the failure mode Proposition 1 forbids. The same discipline sorts mapping errors into two kinds with different fates: a wrong-but-distinct concept costs E nothing (the encoding stays lossless and the error surfaces in the semantic layer that sign-off owns), while a mapping that lands two rows on one key genuinely breaks injectivity, and the reconstruction gate $D(E(s)) = s$ trips on exactly that. Where a filer's printed subtotal differs from (3) because the filer rounded each line independently (observed at SAP: reported subtotals off by one million at millions precision), the row is *demoted to stored* and the discrepancy disclosed: reconstruction stays exact and the filing's arithmetic inconsistency is surfaced rather than hidden. Constructing E's inputs is itself deterministic: the derivation structure comes from the filing's own calculation linkbase on the tagged path and from the verified footing tree on the untagged path (Section 7), the stored-versus-derived classification follows mechanically from it, and no model participates anywhere in the encoding; the only authored ingredients are the one-time, firm-independent simulation layers of Section 8.2.

In a forecast the engine advances the state, $z \mapsto z'$, and the output is rendered through the retained map, $D(z', m)$: exact line-item parity in a forecast is the same mechanism as reconstruction with z' in place of z.

8.2 The state space as a knowledge graph

The union state space is specified as a typed graph over the standard setters' own machine-readable ontologies (the US GAAP and IFRS XBRL taxonomies side by side, roughly 17,000 concepts each) [6, 7]: one graph, each standard's concepts at their own coordinates, values never merged across standards. Nodes carry the attributes the simulator relies on and the taxonomy already provides: period type (instant versus duration, precisely the stock/flow distinction), balance polarity (debit or credit, the sign conventions), monetary flags, and labels. Edges come in five types: *composition* edges imported from the calculation linkbases (subtotal arithmetic, including the newer Calculations 1.1 arcs [5], which one of the four validation filers already uses); *dimensional aggregation* edges for face-of-statement axes; *laws of motion*, authored, recording which flows update which stocks and the cross-statement articulation; *label and synonym* edges, imported, powering the untagged-mode semantic mapper; and *driver attachments* from Section 10. The authored layers cover the simulated core, on the order of one to three hundred concepts, not seventeen thousand; they are the common behavior layer, taken up after the overlay below. Figure 5 draws the machinery on a fragment of the validated filers' overlays; Appendix A draws it complete for one filer, every face line of Apple's three statements with its role and its law of motion.

Encoding a statement builds a *firm overlay* on this graph. Each native line resolves to a node: directly when tagged with a standard concept; as an anchored child of its calculation parent when tagged with a firm extension (SAP's cloud revenue lines are extensions anchored to IFRS revenue); and, in the untagged PDF-only mode, through label-index candidates accepted only if the induced composition subtree foots to the printed subtotals and polarity matches the printed signs, with unresolvable lines flagged under the abstain rule: flag for review, never guess (Section 7). The firm's disclosed leaves become the entries of z; subtotals and aggregates are marked derived and recomputed on decode; m records the exact native presentation on the same overlay. One graph then serves five consumers: extraction disambiguation, the adapters, the engine's stock-flow wiring, driver attachment, and (later) cross-standard correspondence.

The graph's authored layer is *the common behavior layer*, and it is where the fourth design commitment lives. *Behavior* here means how a line moves in simulation: which scenario drivers act on it, which flows carry its changes, and which policy rules may touch it; two lines carrying the same role move by the same rules, whatever their labels or their standard. Concepts answer *which* thing a line is; the behavior layer answers *what kind*, in a vocabulary of ninety authored *economic roles* spanning the three statements: revenue, cost of sales, the operating-expense families, interest income and expense, and tax on the income statement; cash, securities, receivables, inventory, property and equipment, leases, debt, and the equity components on the balance sheet; the cash flow's non-cash add-backs, working-capital deltas, capital expenditure, buybacks, debt issuance and repayment, currency effects, and activity totals. Both standards' concepts map into this one vocabulary, which is the precise sense in which the engine and the driver layer are standard-neutral: they are written once, against roles, never against any standard's or any firm's own lines. Assignment is a deterministic cascade with recorded provenance, so a reviewer can see why every line moves the way it does: a curated concept table over both taxonomies' names decides first; where the concept is unknown, label keywords with the section as context decide; where those are silent, a conservative default by section and polarity holds the line (an unrecognized debit among current assets behaves as an other current asset); and firm extensions inherit through their labels, their anchors, and the calc-descendant rule (an extension behaves as the

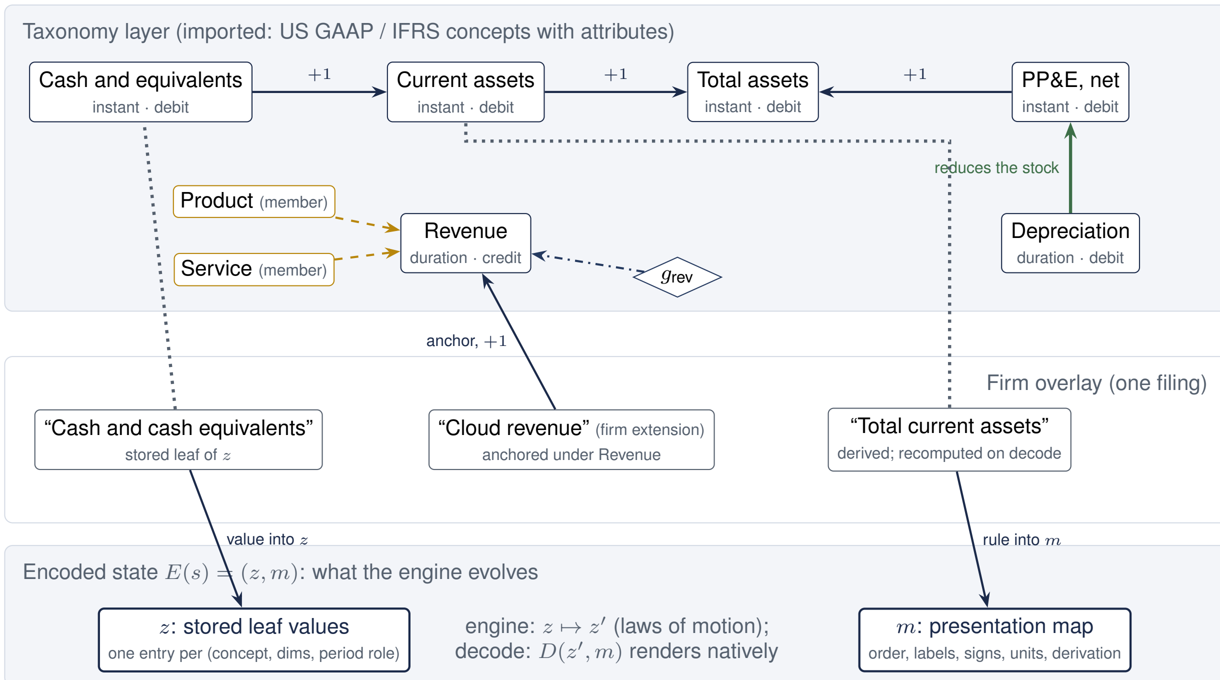

Figure 5. The state space as a typed graph, drawn on a fragment of the validation build's overlays. Nodes are taxonomy concepts carrying the attributes the simulator relies on: period type (instant = stock, duration = flow) and balance polarity. Solid navy arrows are *composition* edges imported from the filing's calculation linkbases, with their weights; dashed gold arrows are *dimensional aggregation* (Apple discloses product and service revenue members on the statement face); the heavy green arrow is an authored *law of motion* (depreciation reduces the capital stock); the dash-dotted arrow is a *driver attachment* from Section 10; dotted slate lines show native lines resolving onto nodes through the imported *label and synonym* index. The overlay beneath shows the three resolution cases from the text: a native line stored as a leaf of z, a firm extension anchored into the calculation subtree (SAP's cloud revenue under IFRS revenue), and a derived subtotal recomputed on decode. The bottom band is the encoding of Section 8.1: z stores one value per disclosed leaf and m stores the presentation and derivation rules, the engine advances only z, and $D(z', m)$ re-renders next-period statements in the same native lines.

subtotal it feeds). In the validation build this cascade classified every face line of all four filers with no per-firm code.

Two boundaries keep the layer honest. The accounting identities need nothing from it: arithmetic is universal, so footing, A = L + E, and the cash tie bind on the firm's own concrete accounts whatever their roles, and a misclassified line can steer a driver wrong but can never unbalance a book. And where classification runs out at the level that matters, the system abstains rather than guesses: a firm whose core driver roles do not resolve is refused simulation with the missing roles named. The naming is deliberate on both sides: the *union* state space is common as a container (one graph both standards inhabit at their own coordinates, values never merged across standards); the *behavior* layer is common as a vocabulary, the one place where a US GAAP line and an IFRS line become the same kind of thing.

Three cautions are built in, all encountered in practice during validation: filed calculation linkbases are treated as the per-firm arithmetic backbone but verified, never trusted (SAP's own subtotals violate them at printed precision); dimensional breakdowns are restricted to face-of-statement axes for the MVP (note-level axes leak into face concepts otherwise); and taxonomy versions are pinned per fiscal year.

8.3 Edges of the vocabulary: two worked examples

The vocabulary's coverage claim is easy to misread as a claim that ninety roles understand every industry. What the cascade actually guarantees is weaker and more useful: every line lands in a safe kind, and the core refuses rather than guesses. Two lines from sectors outside the validation set make the guarantee, and its limits, concrete.

An asset retirement obligation (an oil producer). The obligation to dismantle a rig and restore the site is a discounted liability that grows as its discount unwinds (accretion), settles as wells are decommissioned, and gains new layers as assets are drilled. The cascade has never heard of it: the US GAAP concepts are not in the concept table and no label rule matches, so the section-and-polarity default files the line as an other non-current liability, with that default recorded as its provenance; an IFRS filer that presents it inside decommissioning provisions hits the concept table and lands on the provision role. The MVP treatment is conservative in either case: the stock holds, or moves to a revenue-scaled target where one of the firm's own working-capital cash-flow lines binds it; an accretion row on the cash flow classifies as other non-cash and projects to zero; the capitalized retirement cost inside property and equipment depreciates with the capital pool. Nothing refuses, because the role is not core, and nothing can unbalance, because the identities never consult roles; the approximation is a line that should move and does not, in plain view of the plausibility bounds and the accountant. Accounting for it properly is a vocabulary gap: one new role with concept-table entries for both standards, and one authored law of motion (accretion at the locked discount rate, settlements through the firm's own cash-flow line, new layers arriving with capital expenditure), written once and serving every energy name.

Premiums receivable, net of allowance (an insurer). Here the cascade fails earlier and louder. Before the receivable matters, the income statement's "Premiums earned" matches no revenue rule, the revenue role never resolves, and the firm is refused simulation with the missing roles named: on a misread core the system abstains rather than simulates. The receivable itself, on the unclassified balance sheet insurers commonly present, would default to an other non-current asset and hold. Accounting for it properly is the cheaper kind of gap, a recognizer gap: premiums receivable is the receivable kind the vocabulary already has, tracking premium revenue through its own cash-flow line with the allowance netted inside the line exactly as "Accounts receivable, net" is treated today. A concept-table or label-rule entry mapping premiums earned to the revenue role and premiums receivable to the receivable role restores full behavior; no new role, no new law.

The two examples split coverage failures into their two kinds with their two prices: a recognizer gap costs one mapping line into an existing role; a vocabulary gap costs a new role and a new law. The defaults keep both safe in the meantime, refusal guards the core, and the gating set's deliberately awkward inclusions (Section 12) are the mechanism that forces such firms into the validated set. The honest boundary stands: the validation set is tech and software, and sector-specific dynamics remain unmeasured until such filers enter it.

9. The accounting engine: no plugs, no circularity

The engine's contract is that it *cannot* produce an unbalanced statement, and that nothing is plugged to make that true. Circularity is the companion problem: in a conventional forecast, interest depends on debt, debt on the cash shortfall, and cash on interest (a loop closed by iteration or by a plug); here every quantity is computed once, in an explicit dependency order, so the loop never forms. Let the balance-sheet leaves be accounts $a \in \mathcal{A}$ with balance polarity $\beta_a \in \{+1 \text{ (debit)}, -1 \text{ (credit)}\}$. A *flow* f posts amounts $\phi_f(a)$ to a set of accounts; a flow is *balanced* when its debit-signed legs cancel:

$$\sum_{a\in\mathcal{A}} \beta_a\, \phi_f(a) \;=\; 0. \tag{4}$$

Stocks evolve only through flows, $s_{t+1}(a) = s_t(a) + \sum_f \phi_f(a)$, so by linearity the accounting identity residual

$$R_t \;=\; \sum_a \beta_a\, s_t(a) \;=\; \text{Assets} - \text{Liabilities} - \text{Equity} \tag{5}$$

is invariant: $R_{t+1} = R_t$ for any set of balanced flows. The engine asserts this invariance exactly on every simulated path. The base residual R_0 is zero for three of the four validation filers; for the fourth (SAP) the *printed* leaves round to a two-million-euro imbalance at millions precision (the filer's own rounding), which the engine preserves to the cent and reports rather than absorbing into any account. That distinction (preserve the filing's residual, add exactly zero of your own) is the operational meaning of "no plugs."

Period cycle. Each simulated period runs: (1) project income-statement leaves from the driver draw; (2) set operating-stock targets (receivables, inventory, payables, deferred revenue and their kin) through the firm's *own* working-capital cash flow lines, so a stock moves only if the firm's cash flow statement has a line that carries its delta; (3) hold the discretionary schedule at reasoned levels (capital expenditure scaling with revenue, dividends growing with capped income growth, buybacks policy-scaled, debt issuance and repayment held at the filed schedule, lease payments held with additions imputed); (4) close income to retained earnings, route share-based compensation to paid-in capital, buybacks to treasury where the firm carries one; (5) apply the liquidity policy: excess cash above the operating target sweeps into the firm's own investment-purchase lines (a behavioral treasury rule, not an accounting plug: the books balance with or without it); (6) recompute every derived row through the firm's calculation arcs; (7) assert the invariants: residual invariance, and the cash tie, ending cash equals beginning cash plus the recomputed net-change row of the firm's own cash flow statement.

Articulation. Cross-statement identities hold by construction because both sides come from the same flow: the depreciation add-back equals the amount removed from the capital pool; working-capital rows equal the negatives of their stock deltas; IFRS-style operating add-backs (tax expense, net finance items) mirror the projected income lines with their cash counterparts (taxes paid, interest received and paid) articulated so the accrual-cash gap is zero and the related stocks stay put. The validation build's four filers exercise materially different articulation styles (US GAAP indirect with supplemental cash disclosures; IFRS starting from profit after tax with add-backs and cash items inside the operating total) and both close exactly.

Symbolic verification before numerics. Following the prior interns' frameworks, the engine's flow system is verified *symbolically* before any simulation runs: per firm, every flow's legs are assembled as SymPy expressions over the firm's own accounts, and $\sum_a \beta_a \Delta_a$ must simplify to zero identically, for all parameter values and noise draws, not merely on sampled paths. A nonzero residual names the unbalanced flow symbols, localizing the specification error; the engine's computation order is likewise validated as a directed acyclic graph with an explicit topological order (every quantity computed before anything that uses it), making no-circularity a checked structural property rather than a code-reading exercise. This check earned its place immediately: it exposed a latent gap (income attributable to non-controlling interests posted to cash without an equity counterpart for firms lacking an NCI equity line) that four clean test firms could never trigger numerically. The runtime pipeline is then the interns' sequence exactly: symbolic checking, TensorFlow execution, numerical checking.

Auditability. Every flow is journaled with its magnitude and its double-entry effect description; every run writes the journal, the input hashes, versions, configuration, and seed. Reruns are bit-identical. An auditor can take any simulated figure and walk it back to filed values and named rules.

10. The driver-to-flow layer and scenarios

Scenarios are vectors $x = (\Delta g^{\mathsf{GDP}},\ \Delta\pi,\ \Delta r,\ z_c,\ z_d)$: GDP growth deviation from trend (percentage points), inflation deviation (pp), a parallel short-rate shift (basis points), an industry competition shock (z-score), and a firm demand shock (z-score). The MVP driver map is reasoned and nonlinear where the economics demands it, with parameters documented, not fitted. Writing $\hat{g}$ for the firm's own trailing revenue growth, w_m for the mean-reversion weight applied to it, α for the elasticity of operating expense to revenue, β, λ, κ, and ρ for the sensitivity families (demand, inflation pass-through, competition, and rate pass-through to the firm's asset yields y and debt rates c), and ε for independent Gaussian draws:

$$g_{\mathsf{rev}} = w_m\,\hat{g} \;+\; \tfrac{1}{100}\big(\beta_G\,\Delta g^{\mathsf{GDP}} + \lambda_\pi\,\Delta\pi\big) \;+\; \beta_d z_d \;-\; \kappa_r z_c \;+\; \varepsilon_g, \qquad \varepsilon_g \sim \mathcal{N}(0,\sigma_g^2), \tag{6}$$

$$\Delta\Big(\frac{\mathsf{COGS}}{\mathsf{Rev}}\Big) = \kappa_m z_c \;+\; \lambda_c\,\tfrac{\Delta\pi}{100} \;+\; \varepsilon_m, \qquad g_{\mathsf{opex}} = \alpha\, g_{\mathsf{rev}} + \lambda_o\,\tfrac{\Delta\pi}{100} + \varepsilon_o, \tag{7}$$

$$y' = y + \rho_A\,\tfrac{\Delta r}{10^4}\ \text{(asset yields)}, \qquad c' = c + \rho_D\,\tfrac{\Delta r}{10^4}\ \text{(debt rates)}, \quad \rho_A > \rho_D, \tag{8}$$

with $\lambda_\pi < \lambda_c$ (inflation passes into costs faster than into prices, so a pure cost shock compresses margins); effective tax rate clipped to $[5\%, 45\%]$; dividend growth capped at $\min(g_{\mathsf{rev}}, 10\%)$ and floored at zero; buybacks halved in deep downturns; revenue growth floored at -35%. Monte Carlo runs use common random numbers across scenarios, so scenario mean differences measure the scenario response, not sampling noise. Firms without separate interest lines receive the net rate effect on their combined other-income line, sized by their own cash, securities, and debt balances. Where a driver has no line to act on, it inherits down the concept hierarchy of the knowledge graph; the uniform-rate mechanics of that inheritance and their consequences are laid out in Section 8.1.

For a fixed scenario, Monte Carlo sampling of $(\varepsilon_g, \varepsilon_m, \varepsilon_o)$ propagates through the engine, so the output is a distribution over complete, internally consistent statements; the identity assertions run on every path. The stochastic simulation itself runs in TensorFlow, following the batched-tensor pattern of the internal forecasting stack: all paths of a scenario execute as one vectorized float64 pass over $[n_{\mathsf{paths}}, n_{\mathsf{accounts}}]$ tensors with stateless seeded draws, per-path identity residuals checked to the currency unit; the median and noise-free paths are then replayed bit-exactly through the exact-arithmetic Decimal engine, fed the very shocks TensorFlow drew, to produce the audit artifacts. An agreement test holds the two implementations together at 10^{-10} relative on replayed paths. The **directional battery** requires, on scenario means versus baseline: expansion raises revenue and net income; recession lowers both; competition lowers net income and compresses gross margin; inflation without demand lowers net income; and a rate hike moves net income with the sign of the firm's own net cash position, computed from its balance sheet, not asserted. Part II replaces (6)–(8)'s noise with a learned conditional joint distribution, keeping every other component fixed.

11. Validation evidence: four filers, two standards

Everything in this section was produced by the reference implementation accompanying this proposal (`src/fss`, roughly 9,000 lines of typed Python with a pytest suite and full audit manifests), run end to end on the most recent annual reports of Apple and Microsoft (US GAAP, Form 10-K) and SAP and Spotify (IFRS, Form 20-F). PDFs were rendered from the official EDGAR primary documents; the tag path (Arelle over the filers' inline XBRL) supplies ground truth. The evidence runs in three regimes, each at its own bar: the tag path as ground truth, the PDF-only ablation measured against it, and the truly untagged sweep over arbitrary investor-relations documents carrying no tags at all, six of which are additionally scored against their filers' tagged filings, giving the foreign-layout regime a measured bar of its own. Balance sheet, income statement, and cash flow statement, all reported columns.

Extraction (PDF-only ablation). Zero errors on every accepted cell, all four filers, all three statements, all columns:

Filer (standard)	compared	match	mismatch	missing	flagged
Apple (US GAAP)	198	198	0	0	0
Microsoft (US GAAP)	227	227	0	0	0
SAP (IFRS)	319	319	0	0	0
Spotify (IFRS)	250	250	0	0	0
Total	994	994	0	0	0

Table 2. PDF-only extraction versus tag-path ground truth. "Compared" counts accepted numeric cells; the rule-of-three 95% upper bound on the per-field error rate at 994/994 is 0.30%. Every unmatched ground-truth row falls in a documented benign category (values printed inside a label rather than as cells; derived subtotals the filer does not print, verified through their children).

The pathologies survived on the way to zero are the point of the exercise: wrapped labels with values on either the first or the continuation line; label-embedded amounts ("allowance of $944 and $830"); share counts in thousands beside dollars in millions on one statement; IFRS note-reference columns, both alphanumeric and bare-numeric; unlabeled section-total rows; superscript footnote digits fused to labels and to column years; dashes as explicit zeros; and negated-label sign conventions throughout the cash flows. Each was caught by reader disagreement or ground-truth mismatch during development, fixed as a general rule (never a firm-specific patch), and several are now seeded-error regression tests.

Reconstruction. $D(E(s)) = s$ exactly on all 12 statements, 1,162 cells: values, labels, order, signs, units, precision. SAP's sixteen filer-rounded subtotals were demoted to stored values with the discrepancies disclosed, exactly as Section 8.1 specifies; the other three filers required none.

Footing and identities. Every derived cell of every extracted statement foots within the decimals tolerance; A $=$ L $+$ E and the cash ties hold on all filers (SAP's printed one-million rounding differences pass under the tolerance and are reported).

Simulation. Six scenarios (baseline, expansion, recession, competition, rate hike, inflation), 500 Monte Carlo paths each, per filer: zero identity violations on all 12,000 paths; the engine preserves each filer's base rounding residual exactly and adds none. The directional battery passes in full for all four filers, including the net-cash-position-signed rate test. Representative simulated statements (median-net-income path per scenario), the per-path metric distributions, and the flow journals are written under `out/acceptance/` for accountant review; automated plausibility bounds (growth within $[-40\%, +60\%]$, gross margin within 15pp of base, non-negative cash, positive assets, exact continuity) pass on all representative and deterministic paths.

Untagged sweep (arbitrary IR documents). Beyond the gating set, the untagged pipeline ran over fourteen investor-relations annual reports with no XBRL anywhere: nine current large-caps (including a bank, and IFRS filers from Germany, France, and Hong Kong) and five distressed filers (Lehman 2007, SVB 2022, Bed Bath & Beyond 2022, Wirecard 2018, Evergrande 2022). These documents exhibit pathologies the EDGAR set never shows, each now handled by a general rule with a regression test: DOCX-exported PDFs that split digits inside numbers ("245, 122") and even inside words; PDFs whose space glyphs vanish entirely ("CONSOLIDATEDBALANCESHEET"); pages carrying two statements; statements spanning pages with repeated titles; numeric note-reference columns; bank vocabulary ("statement of financial condition") and IFRS wording ("statement of profit or loss"). Where extraction succeeds, printed-subtotal footing (with the signs and running subtotals of income-statement chains discovered from the printed arithmetic itself), $A = L + E$, and the cash tie verify without any tags, and the discovered structure is itself the calculation tree the simulator uses; a structural quality gate refuses simulation while any balance-sheet cell remains flagged, which is exactly where the LLM adjudicator (or a human reviewer) enters. Page location is overwhelmingly deterministic: 37 of the 42 statement locations across the fourteen mapping artifacts came from the locator alone, and the LLM fallback supplied the remaining five (the cash flow statements of Bed Bath & Beyond and Lehman, and all three statements of Wirecard, whose numeral-stripped fonts leave the locator's density test nothing to count), each choice recorded as located-by provenance in its artifact. One document's English financial pages are image-based, with extractable text only in its Chinese-language half and extraction there partial: the designed boundary for the optional OCR/vision reader. Per-document reports, provenance, and the sweep summary land under `out/untagged/`.

Cross-year carry-forward. Onboarding recurs every fiscal year, so the build also demonstrates carry-forward (Section 7.1): each firm's adjacent-year filing, rendered from its EDGAR primary document, onboarded with and without seeding from the year already onboarded, and every label-to-concept choice scored against the filer's own tags for that year. Microsoft FY2024 from FY2025 is the boundary case: Microsoft is one of the four filers the label lexicon is harvested from, so 80 of its 83 mapped face rows resolve lexically, not one mapped row disagrees with the filer's own tag, carry has nothing left to add, and the only model-resolved rows are the three cash-flow lines Microsoft reworded between the two years, exactly the deltas the design routes to the model. Bed Bath & Beyond fiscal 2021 from the fiscal-2022 artifact is the payoff case: a distressed filer outside the lexicon, where carry resolved 26 label-to-concept choices without any model call (twelve balance-sheet, six income-statement, eight cash-flow), cut model calls from 55 to 37 and model-mapped rows from 35 to 14, and the carried document extracts, foots, ties, and simulates end to end, with the runtime replaying the carried artifact model-free. The scorer sharpened two claims the hard way: the label lexicon was statement-blind ("Inventories" names the stock on a balance sheet and the period's delta on a cash flow), which a statement-scoped lexicon fixed, taking Microsoft's cash-flow concept mismatches from seven to zero; and carry replicates the prior artifact verbatim, errors included (its residual mismatches trace to the prior build's unreviewed shortlist choices, which today mix the two standards' concepts), which is precisely why the artifact's sign-off, not the mechanism, is the accuracy gate. Full tables land under `out/carry/`.

Foreign layouts against their filings' tags. Six of the sweep's investor-relations documents belong to filers whose same-fiscal-year filings are tagged on EDGAR (General Motors, Exxon Mobil, JPMorgan, Alphabet, SVB, and Bed Bath & Beyond), which turns them into a measured extension of the ablation: the same reported figures, printed in a genuinely different layout, scored against the filer's own tags. Each document is replayed deterministically from its committed artifact, and every accepted cell is compared to the tagged value for the same label and fiscal year. The result: 1,343 accepted cells compared, of which 1,316, or 98.0%, equal the filing exactly. The 27 exceptions are the finding, each in a named category: 21 are Exxon's balance sheet, the fused two-statement page, where the document's own correct values land under the wrong year column (its empty first column is likewise all 11 "missing" cells); 3 are JPMorgan's weighted-average share rows read at printed magnitude rather than at the share scale; and 3 are values the documents print where the filing's identically labeled row is empty. Digits read true on foreign layouts; the residual failure mode is layout *attribution* (columns and scale), which no within-document check can see and this cross-document scorer exists to catch, so the two attribution defects enter the Phase 1 worklist as accepted-cell findings under the sev-1 rule (Section 12). The two bank balance sheets do not align at all (the known netted-presentation block: JPMorgan's 108 tagged rows and SVB's 28 find no extracted counterpart), and neither document simulates: the integrity gates refuse JPMorgan on a flagged balance-sheet cell and a failed $A = L + E$, and SVB on unverified cash-flow footing. The same scoring puts an honest number on semantic mapping: of 270 label-to-concept choices with a taggable counterpart, 111 match the filer's tag; the sweep-era artifacts freely chose IFRS concepts and other firms' extensions on US GAAP filers, which is exactly why unreviewed mappings never pass sign-off and why Phase 1 constrains shortlists by standard and filer. Mapping errors corrupt no arithmetic, since concepts play no part in what the readers accept; they gate what may simulate. Full per-document tables land under `out/ir_validation/`.

What this de-risks. The executive summary's four commitments are now demonstrated on real filings rather than argued. Ingestion at the near-zero bar: the decorrelated readers and agreement gate produced zero accepted-cell errors under the PDF-only ablation. Lossless encoding: values and presentation separate exactly, so simulation operates on z alone and re-renders natively, and the substrate claim beneath it holds (the taxonomy graph carries what the simulator needs; real statements land on it without manual mapping). An engine that cannot break the books: the no-plug discipline survives contact with materially different real-world articulation styles under two standards. Machinery written once: the behavior layer's cascade classified every face line of all four filers with no per-firm code, and one scenario definition drove every firm's shape through one engine.

12. Testing and acceptance plan

Testing operates at four levels, all automated except the last:

1. **Unit and property tests**: the number grammar (parentheses, dashes, unicode minus, glued currency, scale clauses), the display grammar (wrapped labels, unlabeled totals, section tracking), the gate's seeded-error battery (a corrupted reader must be flagged or outvoted by independent evidence, never silently accepted; correlated line-reader errors must not outvote positional evidence), encoder injectivity cases (dimensioned rows, period roles, filer-rounded subtotals), and driver response signs.
2. **Golden statement tests**: extracted statements for the gating set are frozen; any change in extraction output diffs against the golden copies cell by cell.
3. **The acceptance battery** (`python -m fss accept`): re-runs extraction accuracy, reconstruction, footing, the full Monte Carlo, directional and plausibility batteries, and writes the acceptance report and manifest. This is the go/no-go gate for any change, and its verdict is binary.
4. **Accountant review**: representative simulated statements per firm per scenario, rendered natively, reviewed under the "not obviously wrong" standard; findings feed the plausibility battery as new automated bounds.

The gating set for the internship phases extends the validation set to roughly three filers per standard with one deliberately awkward inclusion each (a filer with discontinued operations mid-history, and a loss-making filer), plus one truly untagged document (an older or private-company annual report) to exercise the PDF-only semantic mapper without any same-filing metadata. Ground truth for the untagged document is hand-verified double-entry.

Every accepted-cell error found at any point is a sev-1 against the extraction layer: the fix must be a general rule, demonstrated by a new seeded-error test, never a filer-specific patch.

13. Governance, controls, and operations

Model risk alignment (SR 11-7). The system is documented as: purpose and use (Section 3), methodology and theory (Sections 8.1–10), data lineage (filing hashes to outputs), developmental evidence (Section 11), ongoing monitoring (the acceptance battery re-run on every change and on every new filing ingested), and known limitations (Section 15). Effective challenge is enabled by determinism: any reviewer can reproduce any number from the manifest.

Data quality (BCBS 239 posture). Accuracy and integrity are the extraction gates themselves; completeness is measured (coverage and flag rates per statement); timeliness is bounded by EDGAR availability plus pipeline runtime (minutes per filer); adaptability is the role-table and rule architecture, changes to which are code-reviewed and battery-gated.

Audit trail. Per run: SHA-256 of every input document, tool and package versions, configuration, seeds, per-field provenance (each reader's raw token, the rule that accepted or flagged it), the encode demotion list, the flow journal per simulated path, and the validation verdicts. Retention follows the bank's records schedule; nothing in the pipeline mutates its inputs.

Security and licensing. The runtime path is fully local; its only egress is fetching public filings from EDGAR with the mandated identification headers and rate limits. Build-time onboarding may additionally call the hosted LLM endpoint under the leash of Section 7, sending only excerpts of the public document being onboarded; no model is reachable at run time, and with no endpoint configured every build stage degrades to its deterministic behavior with the affected cells left flagged. Dependencies are permissively licensed (Apache-2.0, MIT, BSD); no copyleft components. Taxonomy files are cached under their publishers' terms and not redistributed. The architecture neither requires nor trusts the model: it has no authority to accept a field alone and can never introduce a number no deterministic reader read.

Operations. The pipeline is a CLI (`fetch`, `extract`, `measure`, `accept`, `onboard`, `runtime`) suitable for scheduled batch execution; per-filer runtime is minutes on commodity hardware, dominated by document parsing. Failure modes are loud: a filing that cannot be located, parsed, or reconciled produces a structured error and no partial output.

14. Phased build plan and timeline

The validation build removes risk from the phased plan: what would otherwise be Phase 1's core uncertainty is already reference code. The internship phases harden and extend it to the full bar.

- **Phase 0 (2 weeks): scope lock and harness industrialization.** Fix the gating set (three filers per standard, one untagged document); freeze acceptance criteria with the director and the accountant reviewer; port the validation build's battery into the team's CI; golden statements recorded.
- **Phase 1 (7 weeks): extraction and adapters at the full bar.** Extend the readers (vision reader; prior-year comparatives as an additional path; the LLM semantic mapper for the untagged document, its hypotheses accepted only where footing and polarity confirm them, per Section 8.2); carry-forward onboarding extended from the validated adjacent-year pairs to the full gating set, with mapping shortlists constrained by standard and filer; adjudication UI for flagged fields; the seeded-error battery grown to the full pathology catalogue; the foreign-layout

accuracy bar extended from the preliminary six-document measurement to the full gating set, starting from the two attribution findings that measurement surfaced (a fused-page column displacement and unscaled share rows); IFRS coverage hardened beyond the two validated filers. Gate: the two extraction gates of Section 5 on the full gating set, including the untagged document at its own measured bar.

- **Phase 2 (2 weeks): engine integration and manual toggles.** Wire the encoded state into the engine lineage the team maintains (reconciling with the prior interns' engine where it adds capability); expose flow parameters as reviewed toggles; end-to-end demo on the gating set. This phase is integration, made fast by the validated reference.
- **Phase 3 (3 weeks): drivers, scenarios, and review.** Macro layer consumed from the existing external model (Jodie's) with the documented interface; the directional battery extended with the accountant's scenario expectations; stochastic runs reviewed; plausibility bounds tightened from findings.
- **Hardening (4 weeks)**: accountant sign-off cycles, adversarial filings (restatements, 52/53-week years, currency redenominations), performance, documentation to the governance standard of Section 13.
- **Phase 4 (5 weeks, conditional): learned driver distributions.** The Part II estimator, trained across firms in the union state space, validated on calibration; started only on MVP sign-off.

Phase	Focus	Window	Weeks
Phase 0	Scope lock, harness industrialization	31 Aug to 11 Sep 2026	2
Phase 1	Extraction and adapters at the full bar (US GAAP + IFRS + untagged)	14 Sep to 30 Oct 2026	7
Phase 2	Engine integration and manual flow toggles	2 Nov to 13 Nov 2026	2
Phase 3	Driver layer, scenarios, stochastic validation	16 Nov to 4 Dec 2026	3
Hardening	Accountant review cycles, adversarial filings, docs	7 Dec 2026 to 15 Jan 2027	$\sim$4
Phase 4	Learned driver distributions (conditional)	18 Jan to 26 Feb 2027	$\sim$5

Table 3. Timeline across the internship window. The validation build removes roughly two weeks of risk from Phase 1 and reallocates the time to the untagged-document bar and hardening.

Key milestones. MVP feature-complete by 4 December 2026; accountant sign-off by 15 January 2027; Phase 4 begins only on sign-off.

15. Risk register

Risk	Rating	Mitigation / honest statement
Uniform mis-scaling in untagged mode (identity-invariant)	Medium	Share/EPS cross-checks break scale invariance; prior-year comparatives as a reader; flagged, not silent, when detectors disagree. Residual risk stated in every output.
Filer layouts outside the grammar (new pathology)	Medium	The grammar failed loudly, never silently, on every pathology met so far; flag rate is the alarm; each fix lands as a general rule plus a seeded test.
Driver realism judged insufficient by reviewer	Medium	Part I bar is directional credibility, not point accuracy; parameters documented and adjustable; Part II is the calibrated upgrade path.
Filing anomalies (restatements, 53-week years, redenominations)	Medium	Hardening-phase battery; statements are period-keyed so anomalies surface as explicit mismatches.
Dependence on prior interns' engine internals	Low	The validated reference engine stands alone; integration reconciles capabilities rather than assuming them.
Accountant availability for the human gate	Low	Automated plausibility battery keeps the loop moving; review batches scheduled in the hardening window.

Table 4. Principal risks. Extraction correctness is deliberately absent as a line item: it is the system's central control, gated continuously, not a residual risk to accept.

16. Open questions for alignment

The standards scope (US GAAP and IFRS), the precision asymmetry, exact line-item parity, the state-space specification, and the feasibility of the acceptance gates are settled, the last of these by evidence. Three items remain open:

1. The external macro model's (Jodie's) output interface and granularity, to be confirmed before Phase 3 wiring.
2. The reviewer and counts for the accountant gate: a provisional definition is in place (each gating-set firm, each scenario, representative paths); the named reviewer and batch sizes need confirmation.
3. Which mode gates Part I acceptance: tagged public filings or the untagged born-digital document at the same bar. The pipeline measures both; the recommendation is to gate on tagged filings and report the untagged bar alongside, tightening it to parity during hardening.

A. Appendix: one firm complete, every line wired

Figure 5 showed the state-space machinery on a fragment. Figure 6 removes the elision for one filer: every face line of Apple's balance sheet, income statement, and cash flow statement from the validation build, in the filer's own order, with the layers this proposal builds drawn together. Each boxed line is a row of the encoded state: solid boxes are stored leaves of z; dashed boxes are derived rows, which m records and decode recomputes through Apple's own calculation arcs; gold boxes are the product and service member rows behind the dimensional aggregation of Section 8.1; small-caps lines are the filer's section headers. Each panel carries three aligned columns: the filer's native line; the economic role the deterministic cascade assigned, which is the behavior layer of Section 8.2 at work; and the law of motion the engine of Section 9 applies to that role, written in the driver notation of Section 10. The colored trunks between the cash flow and the balance sheet are the laws of motion as balanced flows: working-capital rows move their bound stocks to driver targets; the held financing schedule moves the debt stocks; capital expenditure and depreciation move the capital pool; the equity flows close income to retained earnings and carry dividends, buybacks, and share-based compensation; the investment lines and the liquidity sweep move the securities stocks; and the recomputed net change, alone, moves cash. Dotted lines from the income statement are articulation: cash-flow rows that mirror projected income lines, which is why the accrual-to-cash gap is zero by construction.

The figure is generated, not drawn: `scripts/gen_apple_overlay_figure.py` reads the validation build's extraction artifacts, classifies every row with the production role cascade, binds working-capital rows to stocks with the production binder, and emits the drawing; the law-of-motion column restates the engine's dispatch, which the acceptance battery gates. Two reading notes. The close of net income to retained earnings is drawn from the cash flow statement's net-income row for legibility; in the engine the close is its own flow, and the cash-flow row is the articulation of the same number. And "held" is a statement about the MVP driver map, not about the architecture: a held entry marks exactly the slot where a Phase 3 reviewed toggle or a Part II learned distribution attaches.

References

[1] I. Vélez-Pareja and J. Tham, "Forecasting Financial Statements with No Plugs and No Circularity," *The IUP Journal of Accounting Research and Audit Practices*, 2010; and I. Vélez-Pareja, "To Plug or Not to Plug: A Note on Financial Statement Forecasting," SSRN Working Paper.

[2] W. Godley and M. Lavoie, *Monetary Economics: An Integrated Approach to Credit, Money, Income, Production and Wealth*, Palgrave Macmillan, 2007.

[3] S. Penman, *Financial Statement Analysis and Security Valuation*, 5th ed., McGraw-Hill, 2013.

[4] XBRL International, *Extensible Business Reporting Language (XBRL) 2.1*, Recommendation, 2003 (with errata).

[5] XBRL International, *Calculations 1.1*, Recommendation, 2023.

[6] Financial Accounting Standards Board, *US GAAP Financial Reporting Taxonomy*, annual releases.

[7] IFRS Foundation, *IFRS Accounting Taxonomy*, annual releases.

[8] U.S. Securities and Exchange Commission, *Interactive Data to Improve Financial Reporting*, Release No. 33-9002, 2009.

[9] U.S. Securities and Exchange Commission, *EDGAR Filer Manual, Volume II*, current version (inline XBRL requirements).

INCOME STATEMENT: drivers act here

native line	*role*	*engine's law of motion*
INCOME STATEMENT		
· Products	revenue	$\times(1+g_{\mathrm{rev}})$
· Services	revenue	$\times(1+g_{\mathrm{rev}})$
Net sales	is_derived	Σ (derived)
· Products	cogs	$\times(1+g_{\mathrm{rev}})(1+\Delta m)$
· Services	cogs	$\times(1+g_{\mathrm{rev}})(1+\Delta m)$
Cost of sales	is_derived	Σ (derived)
Gross margin	is_derived	Σ (derived)
OPERATING EXPENSES:		
Research and development	opex_rnd	$\times(1+g_{\mathrm{opex}})$
Selling, general and administrative	opex_selling	$\times(1+g_{\mathrm{opex}})$
Total operating expenses	is_derived	Σ (derived)
Operating income	is_derived	Σ (derived)
Other income/(expense), net	other_income	$+\Delta y_A$(cash+sec) $-\Delta c_D$ debt
Income before provision for income taxes	is_derived	Σ (derived)
Provision for income taxes	tax	ETR $\times$ pretax (own arcs)
Net income	is_derived	Σ (derived)
EARNINGS PER SHARE:		
Basic (in dollars per share)	eps	parent NI $\div$ shares
Diluted (in dollars per share)	eps	parent NI $\div$ shares
SHARES USED IN COMPUTING EARNINGS PER SHARE:		
Basic (in shares)	share_count	trend $\times$ buyback
Diluted (in shares)	share_count	trend $\times$ buyback

How to read

Each panel has three columns: the filer's native line, the behavior-layer role the deterministic cascade assigned, and the law of motion the engine applies to that role. Solid navy box: stored leaf of z. Dashed slate box: derived row, recomputed through Apple's own calculation arcs on decode. Gold box with ·: a product/service member row (dimensional aggregation). Small-caps lines: the filer's section headers. The laws consume one **driver draw** per Monte Carlo path, the realized set of firm-level driver values (Section 10) for scenario $x = (\Delta g^{\mathrm{GDP}}, \Delta\pi, \Delta r, z_c, z_d)$: g_{rev} revenue growth, Δm COGS-ratio shift, g_{opex} opex growth, $\Delta y_A, \Delta c_D$ rate shifts, g_{div} dividend growth, b buyback factor, noise $\varepsilon_g, \varepsilon_m, \varepsilon_o$; six scenarios, 500 paths each.

- working-capital stock moves
- capital pool: capex, leases, D&A
- debt at the held schedule
- equity flows: close, dividends, buyback, SBC
- securities net purchases and the sweep
- cash tie: net change into the cash stock
- articulation: a cash-flow row mirrors a projected income line

CASH FLOW: articulation and policy

native line	*role*	*engine's law of motion*
STATEMENT OF CASH FLOWS		
Cash, cash equivalents, and restricted cash and cash equivalents, beginning balances	cf_cash_begin	= base cash
OPERATING ACTIVITIES:		
Net income	cf_ni	= projected NI
ADJUSTMENTS TO RECONCILE NET INCOME TO CASH GENERATED BY OPERATING ACTIVITIES:		
Depreciation and amortization	cf_da	$\times(1+g_{\mathrm{rev}})$
Share-based compensation expense	cf_sbc	$\times(1+g_{\mathrm{rev}})$
Other	cf_other_noncash	0
CHANGES IN OPERATING ASSETS AND LIABILITIES:		
Accounts receivable, net	cf_wc	$= -\Delta$ stock
Vendor non-trade receivables	cf_wc	$= -\Delta$ stock
Inventories	cf_wc	$= -\Delta$ stock
Other current and non-current assets	cf_wc	$= -\Delta$ stock
Accounts payable	cf_wc	$= -\Delta$ stock
Other current and non-current liabilities	cf_wc	$= -\Delta$ stock
Cash generated by operating activities	cf_activity_total	Σ (derived)
INVESTING ACTIVITIES:		
Purchases of marketable securities	cf_invest_purchase	held + sweep
Proceeds from maturities of marketable securities	cf_invest_maturity	held; sweep draws
Proceeds from sales of marketable securities	cf_invest_sale	held
Payments for acquisition of property, plant and equipment	cf_capex	$\times(1+g_{\mathrm{rev}})$
Other	cf_other_investing	held
Cash generated by investing activities	cf_activity_total	Σ (derived)
FINANCING ACTIVITIES:		
Payments for taxes related to net share settlement of equity awards	cf_sbc_tax_withhold	$\times(1+g_{\mathrm{rev}})$
Payments for dividends and dividend equivalents	cf_dividends	$\times(1+g_{\mathrm{div}})$
Repurchases of common stock	cf_buyback	$\times\, b$
Proceeds from issuance of term debt, net	cf_debt_issue	held schedule
Repayments of term debt	cf_debt_repay	held schedule
Proceeds from/(Repayments of) commercial paper, net	cf_cp_net	held schedule
Other	cf_other_financing	held
Cash used in financing activities	cf_activity_total	Σ (derived)
Increase/(Decrease) in cash, cash equivalents, and restricted cash and cash equivalents	cf_net_change	Σ (derived)
Cash, cash equivalents, and restricted cash and cash equivalents, ending balances	cf_cash_end	= begin + Δ
SUPPLEMENTAL CASH FLOW DISCLOSURE:		
Cash paid for income taxes, net	cf_tax_paid	= projected tax

BALANCE SHEET: stocks move only through flows

native line	*role*	*engine's law of motion*
STATEMENT OF FINANCIAL POSITION		
ASSETS:		
CURRENT ASSETS:		
Cash and cash equivalents	cash	$= +$ net change (CF)
Marketable securities	securities	+ net purchases
Accounts receivable, net	ar	$\rightarrow \times(1+g_{\mathrm{rev}})$
Vendor non-trade receivables	vendor_receivable	$\rightarrow \times(1+g_{\mathrm{rev}})$
Inventories	inventory	$\rightarrow \times(1+g)(1+\Delta m)$
Other current assets	other_current_asset	$\rightarrow \times(1+g_{\mathrm{rev}})$
Total current assets	bs_derived	Σ (derived)
NON-CURRENT ASSETS:		
Marketable securities	securities	+ net purchases
Property, plant and equipment, net	ppe	+ capex $-$ D&A
Other non-current assets	other_noncurrent_asset	$\rightarrow \times(1+g_{\mathrm{rev}})$
Total non-current assets	bs_derived	Σ (derived)
Total assets	bs_derived	Σ (derived)
LIABILITIES AND SHAREHOLDERS' EQUITY:		
CURRENT LIABILITIES:		
Accounts payable	ap	$\rightarrow \times(1+g)(1+\Delta m)$
Other current liabilities	other_current_liab	$\rightarrow \times(1+g_{\mathrm{rev}})$
Deferred revenue	deferred_revenue	held
Commercial paper	commercial_paper	+ held schedule Δ
Term debt	debt	+ held schedule Δ
Total current liabilities	bs_derived	Σ (derived)
NON-CURRENT LIABILITIES:		
Term debt	debt	+ held schedule Δ
Other non-current liabilities	other_noncurrent_liab	$\rightarrow \times(1+g_{\mathrm{rev}})$
Total non-current liabilities	bs_derived	Σ (derived)
Total liabilities	bs_derived	Σ (derived)
Commitments and contingencies	commitments	(no amount)
Common stock, shares outstanding (in shares)	bs_share_row	held
Common stock, shares issued (in shares)	bs_share_row	held
SHAREHOLDERS' EQUITY:		
Common stock and additional paid-in capital, $0.00001 par value: 50,…	common_stock_apic	+ SBC credited
Accumulated deficit	retained_earnings	+ NI $-$ div $-$ buyback
Accumulated other comprehensive loss	aoci	held
Total shareholders' equity	bs_derived	Σ (derived)
Total liabilities and shareholders' equity	bs_derived	Σ (derived)

Figure 6. Apple's three statements, complete: every face line with its stored or derived kind, its behavior-layer role, and the law of motion the engine applies to it. Colored trunks are the balanced flows that move balance-sheet stocks; dotted lines are articulation from projected income lines.

[10] European Securities and Markets Authority, *European Single Electronic Format (ESEF) Regulatory Technical Standard*, in force 2020.

[11] Arelle project, *Arelle: An Open Source XBRL Platform*, `arelle.org`; version 2.41.6 used in the validation build.

[12] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards Comprehensive Table Extraction from Unstructured Documents," *CVPR*, 2022.

[13] Z. Chen et al., "FinQA: A Dataset of Numerical Reasoning over Financial Data," *EMNLP*, 2021.

[14] F. Zhu et al., "TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance," *ACL*, 2021.

[15] Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference," 2025.

[16] Board of Governors of the Federal Reserve System and OCC, *Supervisory Guidance on Model Risk Management*, SR Letter 11-7, 2011.

[17] Basel Committee on Banking Supervision, *Principles for Effective Risk Data Aggregation and Risk Reporting* (BCBS 239), 2013.

[18] B. D. Jovanovic and P. S. Levy, "A Look at the Rule of Three," *The American Statistician*, 51(2), 1997.

[19] J. Singer-Vine and contributors, *pdfplumber* (MIT license); and pypdf contributors, *pypdf* (BSD license): the two independent PDF engines of the validation build.

[20] KG Encoding Spike report, this repository, June 2026: taxonomy-graph loading, balance-sheet overlay, footing, and round-trip on one filing.